



ЛЕКЦІЯ №3

УМОВНІ ПЕРЕХОДИ

Відношення, логічні операції, умовний перехід, вибір

ПЛАН

1. Булевий тип даних
2. Відношення між величинами
3. Булеві операції
4. Конструкція розгалуження **if-elif-else**
5. Множинний вибір **match-case**

БУЛЕВИЙ ТИП ДАНИХ

Булеві значення представляють одне з двох значень: **True** або **False**.

1. Операції порівняння.

Коли ви порівнюєте між собою числа, рядки чи інші об'єкти:

`5 > 3` → `True` (оскільки 5 більше за 3)

2. Перевірка приналежності або наявності (оператори `in`, `is`).

`"a" in "apple"` → `True` (символ 'a' є в рядку)

3. Логічні операції (`and`, `or`, `not`).

При об'єднанні кількох умов:

`not (10 == 10)` → `False` (інверсія істини)

4. Функції перевірки (методи рядків, типів даних тощо)

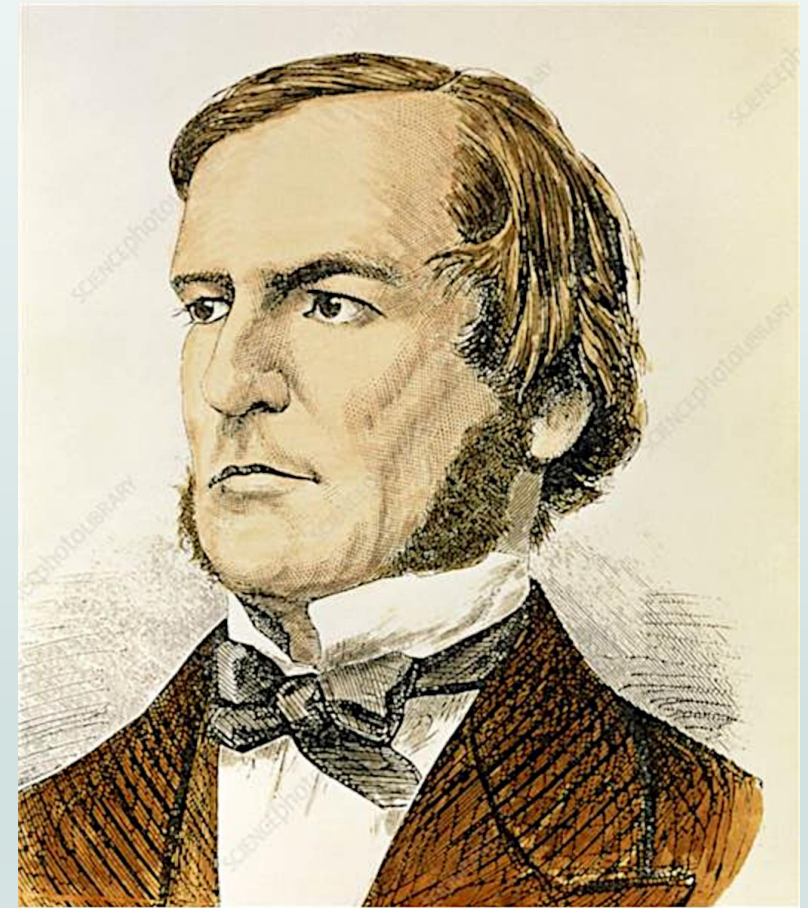
Багато вбудованих функцій у Python створені спеціально для того, щоб повертати

`True` або `False`:

`"12345".isdigit` → `True` (рядок складається лише з цифр)

ІСТОРИЧНА ДОВІДКА

Булевий тип даних названо на честь англійського математика **Джорджа Буля** (George Boole), який у 1854 році у своїй фундаментальній праці «Дослідження законів мислення» описав систему логічних обчислень. У цій системі будь-які твердження зводяться до двох базових станів: істина чи хибність.



ІСТОРИЧНА ДОВІДКА

Етапи впровадження у програмування

- **1937 рік:** Американський інженер Клод Шеннон у своїй магістерській дисертації довів, що булеву алгебру можна використовувати для проєктування електронних схем і комутаторів. Це відкрило шлях до створення цифрових комп'ютерів.
- **1960 рік:** Мова програмування **ALGOL 60** стала першою популярною мовою високого рівня, яка офіційно впровадила булевий тип даних (Boolean) та відповідні логічні оператори.
- **Епоха мови C:** У ранніх версіях мови C окремого логічного типу не існувало. Для умов використовувалися звичайні цілі числа: нуль (0) інтерпретувався як хибність (False), а будь-яке ненульове значення як істина (True). Офіційний тип `bool` з'явився у стандарті C99 завдяки підключенню бібліотеки `<stdbool.h>`.
- **Сучасні мови:** У таких мовах, як Python, Java чи C++, булевий тип (`bool`) є базовим і оперує чіткими константами **True** та **False**.

ВІДНОШЕННЯ МІЖ ВЕЛИЧИНАМИ

Основні оператори відношення у Python:

- **Дорівнює (==):** Перевіряє, чи рівні між собою два операнди.
Наприклад, вираз `5 == 5` поверне `True`, а `5 == 3` – `False`.
- **Не дорівнює (!=):** Перевіряє, чи не рівні між собою два значення.
Вираз `7 != 3` дасть `True`, оскільки числа відрізняються.
- **Більше (>):** Визначає, чи є число зліва більшим за число справа.
Наприклад, `10 > 4` поверне `True`.
- **Менше (<):** Перевіряє, чи є лівий операнд меншим за правий.
Наприклад, `3 < 7` дасть `True`.
- **Більше або дорівнює (>=):** Повертає `True`, якщо значення зліва більше або дорівнює значенню справа.
Наприклад, вираз `5 >= 5` є істинним (`True`).
- **Менше або дорівнює (<=):** Повертає `True`, якщо ліве число менше або дорівнює правому.

ПРІОРИТЕТИ ВИКОНАННЯ ОПЕРАЦІЙ

Рівень	Операції	Опис / Приклади
1	(...)	Круглі дужки (найвищий пріоритет)
2	**	Піднесення до степеня
3	+x, -x, ~x	Унарний плюс, унарний мінус, побітове НЕ
4	*, /, //, %	Множення, ділення, цілочисельне ділення, залишок
5	+, -	Додавання і віднімання
6	<<, >>	Побітові зсуви вліво та вправо
7	&	Побітове І (AND)
8	^	Побітове виключне АБО (XOR)
9		Побітове АБО (OR)
10	<, <=, >, >=, ==, !=	Оператори порівняння
11	is, is not, in, not in	Перевірка ідентичності та приналежності
12	not	Логічне заперечення (НЕ)
13	and	Логічне І
14	or	Логічне АБО
15	= та похідні (+=, -= тощо)	Оператори присвоєння (найнижчий пріоритет)

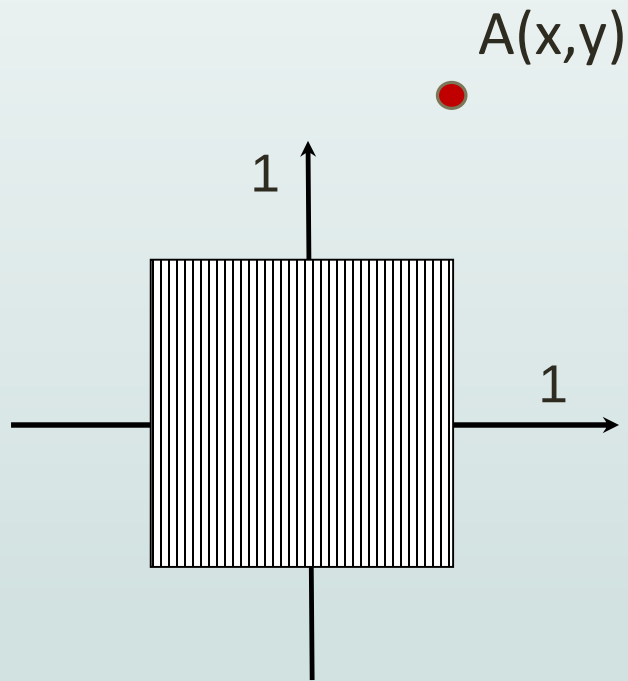
БУЛЕВІ ОПЕРАЦІЇ

Для побудови складних умов використовуються логічні операції:

- **and (логічне І):** повертає True, якщо **обидві** умови істинні.
- **or (логічне АБО):** повертає True, якщо **хоча б одна** умова істинна.
- **not (логічне НЕ):** інвертує значення умови (змінює True на False і навпаки).

A	B	A and B	A or B	not (A)
True	True	True	True	False
True	False	False	True	
False	True	False	True	True
False	False	False	False	

БУЛЕВІ ОПЕРАЦІЇ



Наприклад: Записати умову належності точки заштрихованій частині площини.

$$x \geq -1 \text{ and } x \leq 1 \text{ and } y \geq -1 \text{ and } y \leq 1$$

$$-1 \leq x \leq 1 \text{ and } -1 \leq y \leq 1$$

КОНСТРУКЦІЯ РОЗГАЛУЖЕННЯ IF-ELSE

Для керування потоком виконання програми залежно від виконання умови використовують умовний оператор:

Python

```
if умова:  
    # блок коду, якщо умова істинна  
else:  
    # блок коду, якщо умова хибна
```

```
a = 200  
b = 33  
if b > a:  
    print("b is greater than a")  
else:  
    print("b is not greater than a")
```

КОНСТРУКЦІЯ РОЗГАЛУЖЕННЯ IF-ELSE

Для керування потоком виконання програми залежно від виконання умов використовують умовний оператор:

Python

```
if умова_1:  
    # блок коду, якщо умова_1 істинна  
elif умова_2:  
    # блок коду, якщо умова_2 істинна  
else:  
    # блок коду, якщо всі попередні умови хибні
```

МНОЖИННИЙ ВИБІР MATCH-CASE

match предмет_перевірки:

case шаблон_1: # блок коду для першого шаблону

case шаблон_2 if умова: # блок коду з додатковою умовою (стражем)

case шаблон_3 | шаблон_4: # блок коду для кількох альтернативних шаблонів

case _: # блок коду за замовчуванням (якщо жоден шаблон не підійшов)

МНОЖИННИЙ ВИБІР MATCH–CASE

Основні компоненти синтаксису:

- **match і предмет_перевірки:** Ключове слово `match` приймає вираз, значення якого перевіряється.
- **case і шаблони:** Кожна гілка починається з `case` та визначає шаблон (число, рядок, тип, структуру даних тощо), з яким порівнюється предмет.
- **Оператор альтернативи (|):** Дозволяє вказати кілька варіантів у межах одного `case` (наприклад, `case 1 | 2 | 3:`).
- **Умова-страж (if):** Додатковий логічний вираз після шаблону, який обмежує спрацювання гілки.
- **Універсальний шаблон (_):** Знак підкреслення діє як інструкція `else` за замовчуванням, перехоплюючи всі значення, які не підійшли під попередні

МНОЖИННИЙ ВИБІР MATCH-CASE

```
status_code = 404

match status_code:
    case 200 | 201:
        print("Успіх: запит виконано успішно або ресурс створено")
    case 301 | 302 | 304:
        print("Перенаправлення або кешовані дані")
    case 400 | 401 | 403:
        print("Помилка клієнта: невірний запит або немає доступу")
    case 404:
        print("Ресурс не знайдено (Not Found)")
    case 500 | 502 | 503:
        print("Внутрішня помилка сервера або проблеми зі шлюзом")
    case code if 400 <= code < 500:
        print(f"Інша клієнтська помилка з кодом: {code}")
    case code if 500 <= code < 600:
        print(f"Інша серверна помилка з кодом: {code}")
    case _:
        print("Невідомий статус відповіді")
```